MCP For Unity Documentation

Table of Contents

- I. Introduction
- II. Usage
  - 2.1 Editor Usage
  - 2.2 Runtime Usage
- III. Extensibility
  - 3.1 Editor Extension
  - 3.2 Runtime Extension
- IV. Appendix
- V. AI-Assisted Development

I. Introduction

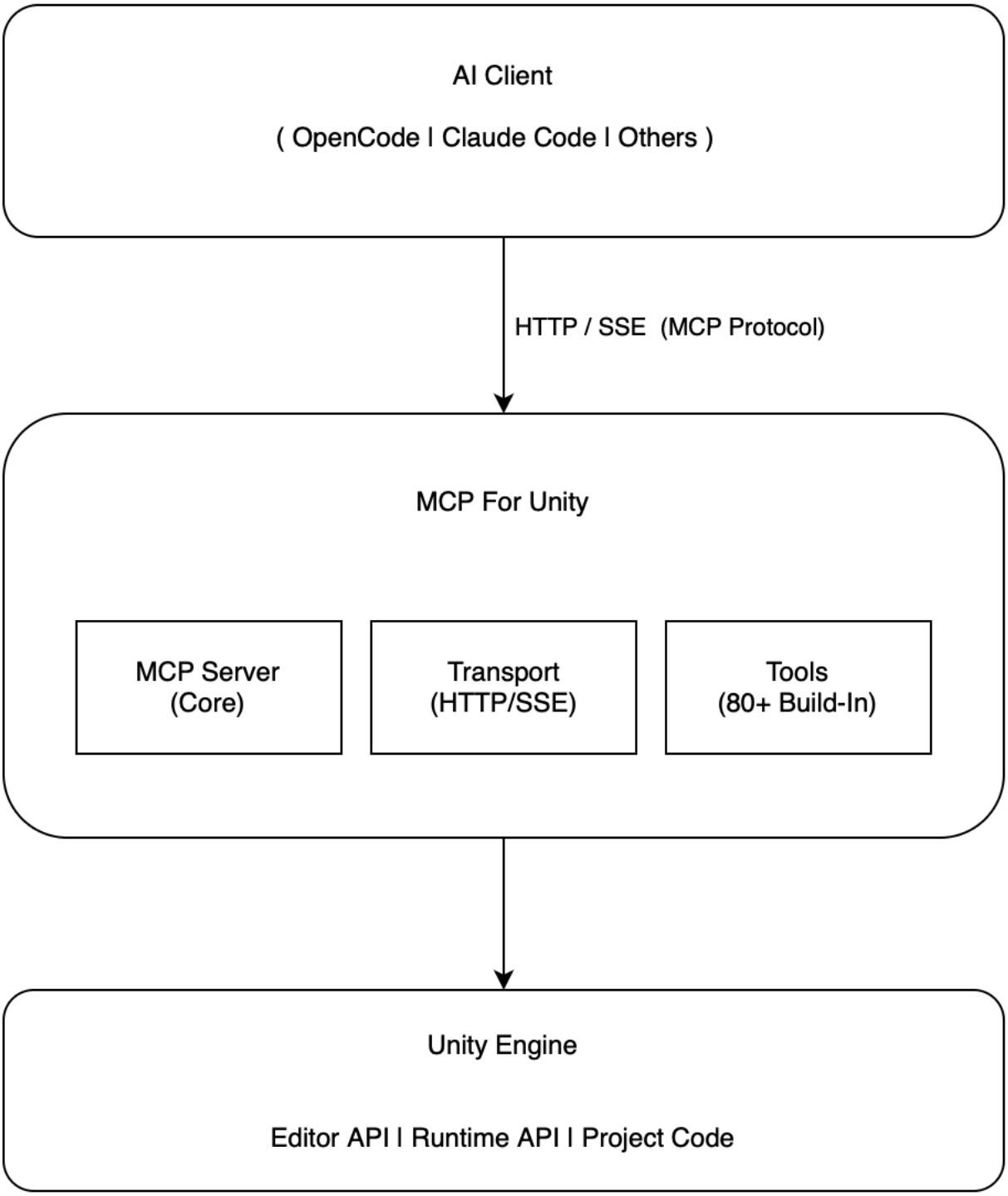
1.1 What is MCP For Unity

MCP For Unity is a Unity implementation framework based on the **Model Context Protocol (MCP)**, designed to seamlessly integrate Unity Editor and runtime applications into the AI ecosystem.

Core Advantages:

Feature	Description
Pure C# Implementation	No external bridges required, no Python or other language dependencies
Full Toolchain Access	AI can directly call Unity Engine APIs and project code
Bidirectional Communication	Supports real-time interaction between AI and Unity
Standardized Protocol	Compatible with Claude Desktop, VS Code MCP, and other mainstream AI clients

1.2 Architecture Overview



1.3 Feature Support

Feature	Support Status
Tools	✅ Fully Supported
Resources	✅ Fully Supported
Tasks	✅ Fully Supported
Prompts	❌ Not Supported

> ⚠️ **Note**: The current version does not support the Prompts feature.

1.4 Technical Specifications

Item	Description
Unity Version	2021.3+
.NET Version	.NET Standard 2.1
Transport Protocol	HTTP + Server-Sent Events (SSE)
Serialization	Newtonsoft.Json
Dependency	`com.unity.nuget.newtonsoft-json`

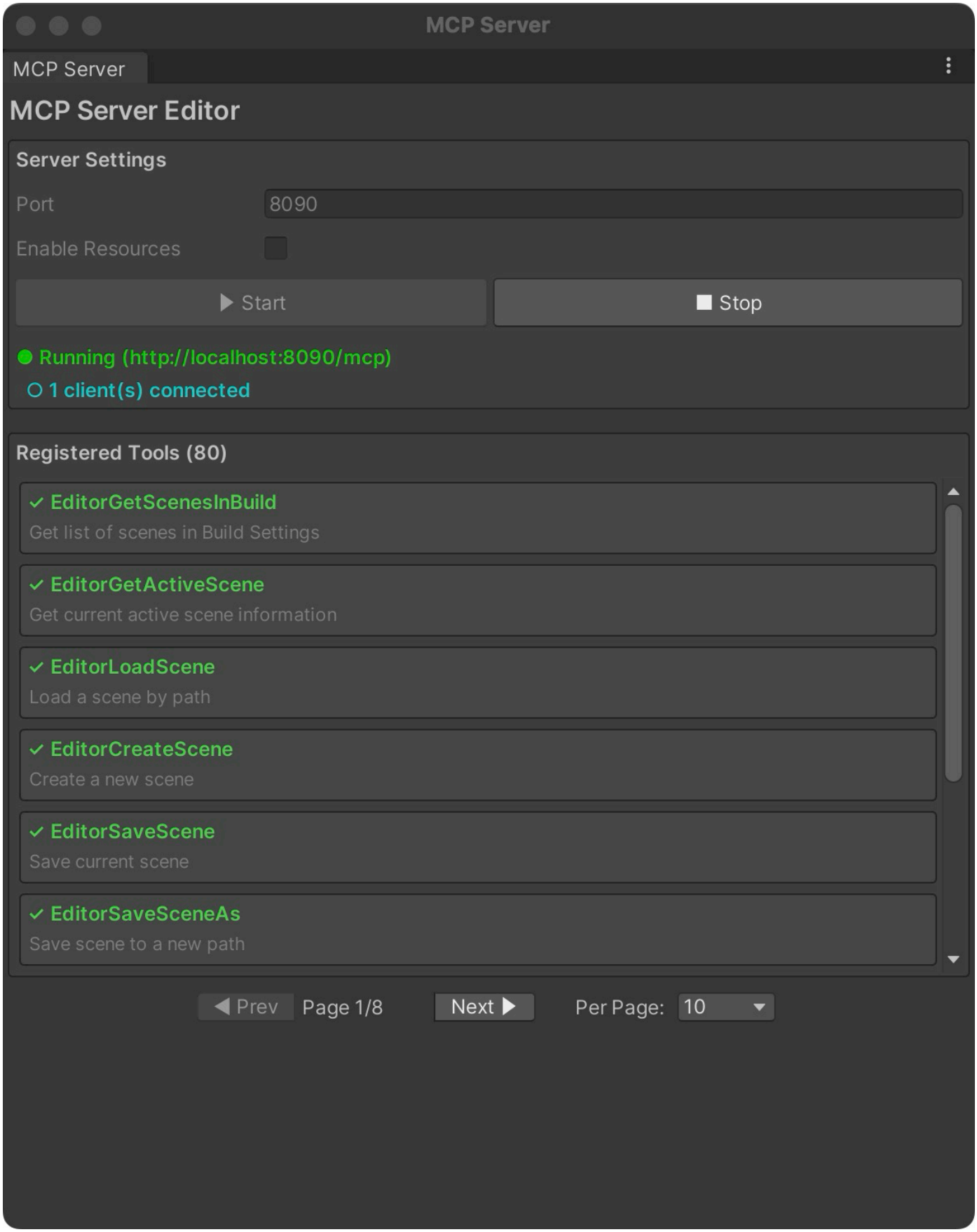
II. Usage

2.1 Editor Usage

2.1.1 Starting the Editor Server

Method 1: Via Editor Window

1. Open menu: `Tools > MCP For Unity > Server Window`
2. Configure server parameters
3. Click **Start** button to launch



Method 2: Via Code

```
using ModelContextProtocol.Editor;

// Start (with default configuration)
GlobalEditorMcpServer.StartServer();

// Stop
GlobalEditorMcpServer.StopServer();
```

2.1.2 Configuration Options

The following options can be configured in the editor window:

Option	Default	Description
Port	8090	Server port
Enable Resources	false	Enable resources service
Enable File Watching	false	Enable file watching

2.1.3 Built-in Tools Overview

MCP For Unity provides **80 editor tools** covering common development scenarios:

Scene Management (10 tools)

Tool Name	Description
`EditorGetScenesInBuild`	Get scene list in Build Settings
`EditorGetActiveScene`	Get current active scene info
`EditorLoadScene`	Load scene
`EditorCreateScene`	Create new scene
`EditorSaveScene`	Save current scene
`EditorSaveSceneAs`	Save scene as
`EditorCloseScene`	Close scene
`EditorAddSceneToBuild`	Add scene to Build
`EditorRemoveSceneFromBuild`	Remove scene from Build
`EditorGetSceneHierarchy`	Get scene hierarchy structure

GameObject Operations (15 tools)

Tool Name	Description
`EditorCreateGameObject`	Create empty GameObject
`EditorCreatePrimitive`	Create primitive geometry
`EditorDuplicateGameObject`	Duplicate GameObject
`EditorDeleteGameObject`	Delete GameObject
`EditorFindGameObject`	Find GameObject
`EditorFindGameObjectsByName`	Find by name
`EditorFindGameObjectsByTag`	Find by tag
`EditorFindGameObjectsByLayer`	Find by layer
`EditorGetGameObjectInfo`	Get detailed info
`EditorGetChildren`	Get children list
`EditorSetParent`	Set parent
`EditorSetTag`	Set tag
`EditorSetLayer`	Set layer
`EditorSetStatic`	Set static flag
`EditorSetActive`	Set active state

Transform Operations (10 tools)

Tool Name	Description
`EditorSetPosition`	Set world position
`EditorSetRotation`	Set world rotation
`EditorSetScale`	Set scale
`EditorSetLocalPosition`	Set local position
`EditorSetLocalRotation`	Set local rotation
`EditorSetLocalScale`	Set local scale
`EditorSetPositionAndRotation`	Set position and rotation
`EditorTranslate`	Relative translation
`EditorRotate`	Relative rotation
`EditorResetTransform`	Reset transform

Component Operations (10 tools)

Tool Name	Description
`EditorAddComponent`	Add component
`EditorRemoveComponent`	Remove component
`EditorGetComponent`	Get component info
`EditorGetComponents`	Get all components
`EditorGetComponentsInChildren`	Get components in children
`EditorSetComponentProperty`	Set component property
`EditorGetComponentProperty`	Get component property
`EditorHasComponent`	Check if has component
`EditorGetRequiredComponent`	Get or add component
`EditorSendComponentMessage`	Send message

Compilation & Build (10 tools)

Tool Name	Description
`EditorGetCompilationStatus`	Get compilation status
`EditorIsCompiling`	Is compiling
`EditorGetCompileErrors`	Get compile errors
`EditorGetCompileWarnings`	Get compile warnings
`EditorGetBuildTarget`	Get build target
`EditorSetBuildTarget`	Set build target
`EditorGetScriptingBackend`	Get scripting backend
`EditorGetAssemblyDefinitions`	Get asmdef list
`EditorGetDefineSymbols`	Get define symbols
`EditorSetDefineSymbols`	Set define symbols

Asset Management (15 tools)

Tool Name	Description
`EditorFindAssets`	Find assets
`EditorGetAssetInfo`	Get asset details
`EditorGetAssetPath`	Get asset path
`EditorGetAssetDependencies`	Get asset dependencies
`EditorGetFolderContents`	Get folder contents
`EditorCreateFolder`	Create folder
`EditorDeleteAsset`	Delete asset
`EditorRenameAsset`	Rename asset
`EditorMoveAsset`	Move asset
`EditorCopyAsset`	Copy asset
`EditorRefreshAssets`	Refresh asset database
`EditorReimportAsset`	Reimport asset
`EditorGetAssetImportSettings`	Get import settings
`EditorLoadAssetAtPath`	Load asset at path
`EditorGetAllAssetPaths`	Get all asset paths

Prefab Operations (10 tools)

Tool Name	Description
`EditorInstantiatePrefab`	Instantiate prefab
`EditorCreatePrefab`	Create prefab
`EditorApplyPrefab`	Apply prefab changes
`EditorRevertPrefab`	Revert prefab
`EditorUnpackPrefab`	Unpack prefab
`EditorGetPrefabInfo`	Get prefab info
`EditorGetPrefabType`	Get prefab type
`EditorIsPrefabInstance`	Is prefab instance
`EditorGetPrefabAssetPath`	Get prefab asset path
`EditorGetAllPrefabs`	Get all prefabs

2.1.4 Implementation Location

```
Assets/McpForUnity/Editor/
├── Tools/
│   └── EditorToolsList.cs      # 80 editor tools implementation
├── GlobalEditorMcpServer.cs    # Global server management
└── Window/
    └── McpServerEditorWindow.cs # Editor window
```

2.1.5 Connecting AI Clients

Claude Desktop Configuration

Edit Claude Desktop configuration file:

- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
- Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "unity-editor": {
      "url": "http://localhost:8090/mcp",
      "transport": "sse"
    }
  }
}
```

OpenCode Configuration

Create `.opencode/opencode.jsonc` file in project root:

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "UnityMCPServer": {
      "type": "remote",
      "url": "http://localhost:8090/mcp",
      "enabled": true
    }
  }
}
```

2.2 Runtime Usage

2.2.1 Starting the Runtime Server

```
using System.Threading.Tasks;
using ModelContextProtocol.Unity;
using UnityEngine;
```

```
public class MCPManager : MonoBehaviour
{
    private McpServerHost _host;

    async void Start()
    {
        // Create configuration
        var options = new McpServerHostOptions
        {
            Port = 3000,
            ServerName = "UnityMCP",
            ServerVersion = "1.0.0"
        };

        // Create and start server
        _host = new McpServerHost(options);
        await _host.StartAsync();

        // Register custom tools
        _host.Server.RegisterToolsFromClass(typeof(MyGameTools));

        Debug.Log($"MCP Server started at http://localhost:{options.Port}/mcp");
    }

    async void OnDestroy()
    {
        if (_host != null)
        {
            await _host.DisposeAsync();
        }
    }
}
```

> ⚠️ **Important**: The Runtime server **does not include any built-in tools**. All tools must be registered manually using `RegisterToolsFromClass()` or `AddCustomTool()`.

2.2.2 Configuration Options

Option	Type	Default	Description
`Port`	int	3000	Server port
`ServerName`	string	"UnityMCP"	Server name
`ServerVersion`	string	"1.0.0"	Server version
`Instructions`	string	-	Server instructions
`LogLevel`	LogLevel	Information	Log level

2.2.3 Event Subscription

```
_host.OnServerStarted += () => Debug.Log("Server started");
_host.OnServerStopped += () => Debug.Log("Server stopped");
_host.OnServerError += (error) => Debug.LogError($"Server error: {error}");
```

2.2.4 Registering Custom Tools

```
// Method 1: Register via class
_host.Server.RegisterToolsFromClass(typeof(MyGameTools));

// Method 2: Add Lambda directly
_host.AddCustomTool("my_tool", "Tool description", async (args, ct) =>
{
    string param = args?["param"]?.ToString();
    // Processing logic
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = "Result" }
        }
    };
});
```

III. Extensibility

3.1 Editor Extension

3.1.1 Defining Custom Tools

Define tools using attribute annotations:

```
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

public static class MyEditorTools
{
    [McpServerTool("EditorMyCustomTool", Description = "My custom editor tool")]
    public static CallToolResult MyCustomTool(
        [McpArgument(Description = "Parameter 1", Required = true)] string param1,
        [McpArgument(Description = "Parameter 2 (optional)")] int param2 = 0)
    {
        // Execute editor operation
        Debug.Log($"Executing custom tool: {param1}, {param2}");

        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"Success: {param1}" }
```

```
            }
        };
    }
}
```

Register tools:

```
// Register after GlobalEditorMcpServer starts
GlobalEditorMcpServer.Server.RegisterToolsFromClass(typeof(MyEditorTools));
```

3.1.2 Parameter Definition Methods

Basic Type Parameters

```
[McpServerTool("Example", Description = "Example")]
public static CallToolResult Example(
    [McpArgument(Description = "String parameter", Required = true)] string text,
    [McpArgument(Description = "Integer parameter")] int number = 0,
    [McpArgument(Description = "Float parameter")] float value = 0f,
    [McpArgument(Description = "Boolean parameter")] bool flag = false)
{
    // ...
}
```

Enum Parameters

```
public enum MyEnum { OptionA, OptionB, OptionC }

[McpServerTool("EnumExample", Description = "Enum example")]
public static CallToolResult EnumExample(
    [McpArgument(Description = "Enum option")] MyEnum option = MyEnum.OptionA)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"Selected: {option}" }
        }
    };
}
```

Array Parameters

```
[McpServerTool("ArrayExample", Description = "Array example")]
public static CallToolResult ArrayExample(
    [McpArgument(Description = "String array", Required = true)] string[] items)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"Received {items.Length} items" }
        }
    };
}
```

Vector3 Parameters (Expanded Form)

```
[McpServerTool("VectorExample", Description = "Vector3 example")]
public static CallToolResult VectorExample(
    [McpArgument(Description = "Position")] UnityEngine.Vector3 position)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"Position: {position}" }
        }
    };
}
```

Parameter format when calling:

```
{
  "position_x": 1.0,
  "position_y": 2.0,
  "position_z": 3.0
}
```

3.1.3 Custom Type Definition

Defining Custom Types

```
using Newtonsoft.Json;
using ModelContextProtocol.Server;

public class PlayerData
{
    [JsonProperty("playerName")]
    [McpArgument(Description = "Player name", Required = true)]
    public string PlayerName;

    [JsonProperty("level")]
```

```
    [McpArgument(Description = "Player level")]
    public int Level;

    [JsonProperty("health")]
    [McpArgument(Description = "Health")]
    public float Health;

    [JsonProperty("position")]
    [McpArgument(Description = "Position")]
    public UnityEngine.Vector3 Position;
}
```

Using Custom Types in Tools

```
[McpServerTool("SetPlayerData", Description = "Set player data")]
public static CallToolResult SetPlayerData(
    [McpArgument(Description = "Player data", Required = true)] PlayerData data)
{
    Debug.Log($"Player: {data.PlayerName}, Level: {data.Level}");

    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"Player set: {data.PlayerName}" }
        }
    };
}
```

Call Format:

```
{
  "data": {
    "playerName": "Player1",
    "level": 10,
    "health": 100.0,
    "position_x": 0,
    "position_y": 1,
    "position_z": 0
  }
}
```

Custom Type Validation Rules

Rule	Description
Fields must have both `[JsonProperty]` and `[McpArgument]`	Fields missing either attribute will be ignored
Use `[JsonRequired]` or `[McpArgument(Required = true)]`	Mark required fields
Supports nested custom types	Recursive validation
Supports custom type arrays	`PlayerData[]` or `List<PlayerData>`

3.1.4 Instance Tool Registration

Instance tools allow registering independent tool sets for multiple instances of the same class.

Defining Instance Tool Class

```
[McpInstanceTool(Name = "Player", Description = "Player instance tools")]
public class PlayerTools
{
    public string PlayerId { get; set; }
    public int Health { get; set; }
    public int Score { get; set; }

    [McpServerTool(Description = "Get player info")]
    public CallToolResult GetInfo()
    {
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"Player {PlayerId}: HP={Health}, Score=
{Score}" }
            }
        };
    }

    [McpServerTool(Description = "Set health")]
    public CallToolResult SetHealth(
        [McpArgument(Description = "Health value", Required = true)] int health)
    {
        Health = health;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"HP set to {Health}" }
            }
        };
    }
```

```
    [McpServerTool(Description = "Add score")]
    public void AddScore(
        [McpArgument(Description = "Score amount", Required = true)] int amount)
    {
        Score += amount;
    }
}
```

Registering Instance Tools

```
// Create instances
var player1 = new PlayerTools { PlayerId = "P001", Health = 100, Score = 0 };
var player2 = new PlayerTools { PlayerId = "P002", Health = 80, Score = 50 };

// Register to server
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player1, "player_1");
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player2, "player_2");

// Tool name format: {instanceId}.{methodName}
// player_1.GetInfo
// player_1.SetHealth
// player_2.GetInfo
// player_2.SetHealth
```

Unregistering Instance Tools

```
// Unregister all tools for specified instance
GlobalEditorMcpServer.Server.UnregisterInstanceTools("player_1");
```

Important Notes

- Instance ID must be unique
- Server holds instance reference, be aware of memory management
- Unregister when no longer needed

3.2 Runtime Extension

Runtime extension works the same way as editor extension.

3.2.1 Custom Tool Definition

```
public static class GameTools
{
    [McpServerTool("SpawnEnemy", Description = "Spawn enemy")]
    public static CallToolResult SpawnEnemy(
        [McpArgument(Description = "Enemy type")] string enemyType,
        [McpArgument(Description = "Position")] UnityEngine.Vector3 position)
    {
```

```
        // Game runtime logic
        var enemy = SpawnManager.SpawnEnemy(enemyType, position);

        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"Enemy spawned: {enemy.name}" }
            }
        };
    }

    [McpServerTool("GetPlayerStatus", Description = "Get player status")]
    public static CallToolResult GetPlayerStatus()
    {
        var player = GameManager.Instance.Player;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = JObject.FromObject(new
                {
                    health = player.Health,
                    mana = player.Mana,
                    level = player.Level
                }).ToString() }
            }
        };
    }
}
```

Register:

```
_host.Server.RegisterToolsFromClass(typeof(GameTools));
```

3.2.2 Instance Tool Registration

```
// Define
[McpInstanceTool(Name = "Unit", Description = "Unit tools")]
public class UnitTools
{
    public Unit Unit { get; set; }

    [McpServerTool(Description = "Move to target")]
    public CallToolResult MoveTo(
        [McpArgument(Description = "Target position")] UnityEngine.Vector3 target)
    {
        Unit.MoveTo(target);
        return new CallToolResult
        {
            Content = new List<ContentBlock>
```

```
            {
                new TextContentBlock { Text = "Move command sent" }
            }
        };
    }
}

// Register
var unit = new UnitTools { Unit = myUnit };
_host.Server.RegisterToolsFromInstance(unit, "unit_001");

// Unregister
_host.Server.UnregisterInstanceTools("unit_001");
```

IV. Appendix

4.1 McpServerHostOptions Configuration Table

Property	Type	Default	Description
`Port`	int	3000	HTTP service port
`ServerName`	string	"UnityMCP"	MCP server name
`ServerVersion`	string	"1.0.0"	Server version
`Instructions`	string	-	Instructions sent to AI
`LogLevel`	LogLevel	Information	Log output level

4.2 Event List

Event	Signature	Trigger
`OnServerStarted`	`Action`	Server started successfully
`OnServerStopped`	`Action`	Server stopped
`OnServerError`	`Action<string>`	Server error occurred

4.3 Return Result Types

Success Result

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "Operation successful" }
    }
};
```

Error Result

```
return new CallToolResult
{
    IsError = true,
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "Error message" }
    }
};
```

Result with Image

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new ImageContentBlock
        {
            Data = base64ImageData,
            MimeType = "image/png"
        }
    }
};
```

4.4 Important Notes

1. **Thread Safety**: MCP tools execute on background threads. Use `MainThreadDispatcher` or ensure main thread execution when operating Unity objects
2. **Lifecycle**: `McpServerHost` automatically listens to `Application.quitting` and cleans up when application exits
3. **Security**: By default listens on `localhost`, does not accept external connections. Configure carefully for production
4. **Performance**: Heavy tool calls may affect performance, control call frequency reasonably
5. **Error Handling**: Tool internal exceptions are caught and returned as error results, server won't crash

4.5 File Structure

```
Assets/McpForUnity/
├── Core/
│   ├── IMcpServerHost.cs          # Server interface
│   ├── McpServerHost.cs           # Server implementation
│   ├── McpServerHostOptions.cs    # Configuration class
│   ├── McpException.cs            # Exception types
│   ├── Throw.cs                   # Parameter validation
│   └── Protocol/                  # MCP protocol types
├── Server/
│   ├── McpServer.cs               # Core server
│   ├── McpServerOptions.cs        # Server configuration
│   └── Tools/
│       └── Attributes.cs          # Tool attribute definitions
├── Transport/
│   ├── ITransport.cs              # Transport interface
│   └── HttpListenerServerTransport.cs
├── Editor/
│   ├── Tools/
│   │   └── EditorToolsList.cs     # 80 editor tools
│   ├── GlobalEditorMcpServer.cs   # Editor server
│   └── Window/
│       └── McpServerEditorWindow.cs
├── Utilities/
│   ├── UnityLogger.cs             # Logging implementation
│   └── MainThreadDispatcher.cs    # Main thread dispatch
└── Samples/
    └── MCPExampleUsage.cs         # Usage examples
```

4.6 FAQ

Q: What if port is occupied?

A: Change `Port` configuration to another port, or release the occupied port.

Q: Newtonsoft.Json not found?

A: Make sure `com.unity.nuget.newtonsoft-json` package is installed:

```
Window > Package Manager > + > Add package by name
Enter: com.unity.nuget.newtonsoft-json
```

Q: How to access MonoBehaviour in tools?

A: Use `GameObject.Find()` or singleton pattern to get object references in scene.

Q: How to pass complex objects as tool parameters?

A: Define custom types, annotate fields with `[JsonProperty]` and `[McpArgument]`.

V. AI-Assisted Development

MCP For Unity supports AI-assisted development through AI clients. Simply copy the project configuration file to your working directory to let AI assistants understand the project structure and assist with development.

5.1 Quick Start

Step 1: Copy Configuration File

Copy `AGENTS.md` to project root:

```
# macOS / Linux
cp Assets/McpForUnity/AGENTS.md ./

# Windows
copy Assets\McpForUnity\AGENTS.md .
```

Step 2: Launch AI Client

Open the project directory using an AI client that supports MCP protocol (such as OpenCode, Claude Desktop, etc.).

Step 3: Start Development

The AI assistant will automatically read project information from `AGENTS.md`, including:

- Code conventions and naming standards
- Project structure and file locations
- Build and test commands
- API usage methods

5.2 Recommended Tools

Tool	Description
OpenCode	Recommended, native MCP protocol support
Claude Desktop	Requires MCP server configuration
Cursor	Supports MCP extension
VS Code + MCP Plugin	Requires MCP extension installation

5.3 Development Workflow

```
1. AI reads AGENTS.md → Understands project conventions
2. Start MCP Server → AI can call Unity tools
3. Describe requirements → AI generates/modifies code
4. AI calls tools → Validates/tests code
5. Iterate → Complete development
```

5.4 Example Scenarios

Scenario	Example Prompt
Add New Tool	"Add an editor tool to export scene hierarchy to JSON"
Fix Issue	"EditorInstantiatePrefab tool is throwing an error, help me fix it"
Code Refactoring	"Refactor this tool class according to project conventions"
Write Tests	"Write unit tests for EditorCreateGameObject"
Add Documentation	"Add comments and usage documentation for the new tool"

5.5 Important Notes

- AI assistant can directly operate Unity Editor through MCP protocol
- Recommend running compilation check after AI modifies code
- For complex operations, proceed step by step for easier verification and rollback

---

Document Version: 1.0.3
Last Updated: 2026-03-02